

Programação Orientada a Objetos (POO)

Programação Back-End

Conceitos fundamentais para organização, segurança e manutenção de sistemas.

Objetivo

Compreender POO, encapsulamento, abstração, interfaces e suas aplicações no Back-End.

POO

**Organização
+ proteção +
manutenção**

O que é Programação Orientada a Objetos?

Definição

POO é uma forma de organizar programas usando objetos. Cada objeto reúne dados e ações relacionadas a ele.

A ideia é representar elementos do problema como entidades que possuem características e comportamentos.

OBJETO

DADOS**AÇÕES**

características

comportamentos

Modelo mental: elementos do sistema → objetos

Classe e objeto

CLASSE

Funciona como um modelo ou molde que define a estrutura e os comportamentos dos objetos.

Molde → define

OBJETO

É uma instância criada a partir de uma classe.

Criação → representa

Exemplo: classe = molde de bolo | objeto = bolo produzido

Atributos, métodos e construtor

ATRIBUTOS

Informações ou características que pertencem ao objeto.

MÉTODOS

Ações ou comportamentos que o objeto consegue executar.

CONSTRUTOR

Função executada na criação do objeto para definir valores iniciais.

Para memorizar

Atributo = característica

Método = ação

Construtor = inicializa

Encapsulamento

Proteção dos dados

Encapsulamento é a prática de proteger e controlar o acesso ao interior de um objeto.

Sem ele, qualquer parte do programa poderia alterar dados de forma inadequada.

CONTA BANCÁRIA

saldo **R\$ 1.000.000**

depositar(valor)

sacar(valor)

Operações passam por regras antes de alterar o estado.

Modificadores de acesso

public**Qualquer parte do sistema**

Acesso aberto

private**Somente a própria classe**

Maior proteção

protected**Classe e classes filhas**

Acesso por herança

Exemplo: um atributo como `#saldo` pode ser privado e manipulado por métodos públicos controlados.

Encapsulamento em um exemplo de Back-End

```
class ContaBancaria {  
  #saldo = 0;  
  
  depositar(valor) {  
    if (valor > 0) this.#saldo += valor;  
  }  
  
  sacar(valor) {  
    if (valor > 0 && valor <= this.#saldo)  
      this.#saldo -= valor;  
  }  
}
```

Por que isso é útil?

- O saldo permanece protegido.
- As operações seguem regras.
- O objeto controla seu próprio estado.

Abstração

CONCEITO

Abstrair é destacar o que é importante para usar um componente e esconder detalhes que não precisam ser conhecidos naquele momento.

Exemplo: dirigir um carro

O motorista utiliza volante e pedais sem precisar conhecer toda a engenharia do motor.

Abstração = usar uma interface simples sem conhecer toda a complexidade interna.

Interface: um contrato

Definição

Uma interface estabelece quais ações devem existir, sem determinar como cada implementação irá executá-las.

Contrato = padrão de comunicação

PAGAMENTO

pagar()

Pix

Cartão

Boleto

Todos possuem a ação pagar(), mas cada um pode implementá-la de forma diferente.

Observação: JavaScript não possui interfaces nativas como Java e TypeScript; podemos aplicar a ideia de contrato.

Como os conceitos se complementam

ENCAPSULAMENTO

Protege dados e controla o acesso.

ABSTRAÇÃO

Esconde complexidade e simplifica o uso.

INTERFACE

Define um padrão de comunicação.

Resultado

Código mais organizado • manutenção mais simples • menor acoplamento

No Back-End, a lógica interna de um serviço pode mudar sem exigir alterações em quem utiliza seu método público.

O que devemos levar da apresentação

POO

Organiza o sistema em objetos com dados e ações.

ENCAPSULAMENTO

Protege dados e controla o acesso.

ABSTRAÇÃO

Esconde complexidade e simplifica a interação.

INTERFACE

Estabelece um padrão ou contrato.

PARA DECORAR

Encapsulamento = protege | Abstração = simplifica | Interface = define o contrato

Esses conceitos contribuem para Back-Ends mais seguros, organizados e fáceis de manter.